

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 4. Operators and Expressions

Lecture: 28. Expressions

Section 1: Lecture Summary

The lecture introduces the concept of **expressions** in Python and explains how expressions are composed of **operators** and **operands** to compute values. It emphasizes the importance of understanding **operator precedence** (the order in which operators are evaluated) and **associativity** (the direction in which operators of the same precedence are evaluated). The instructor explains how multiplication has higher precedence than addition, resulting in different outcomes if not careful. The lecture also highlights the **exponentiation operator's** right-to-left associativity, which is unique compared to other arithmetic operators. Finally, it stresses the critical role of **parentheses** in controlling and overriding the default precedence and associativity to ensure expressions evaluate as intended and recommends always using fully parenthesized expressions to avoid errors especially when the programmer is unsure about operator rules.

Section 2: Key Concepts and Explanations

- What is an Expression?

- An expression is a combination of **operands** (variables, literals) and **operators** (such as `+`, `-`, `/`, `**`) that Python evaluates to produce a value.

- Example: `x = a + b * c` where `a`, `b`, `c` are operands and `+`, `*` are operators.

- Operator Precedence

- Precedence determines the order in which different operators in an expression are evaluated.

- Higher precedence operators are evaluated before lower precedence ones.

- The general precedence hierarchy for arithmetic operators (from highest to lowest):

1. Parentheses `()`

2. Exponentiation `\*\*`

3. Multiplication `\*`, Division `/`, Modulo `%`

4. Addition `+`, Subtraction `-`

- Example:

- `2 + 3 \* 5` → multiplication first ($3 * 5 = 15$), then addition ($2 + 15 = 17$).

- Operator Associativity

- When operators have the same precedence, associativity decides the evaluation order.

- For most operators, associativity is **left-to-right** ('LR'):

- Example: `5 + 2 - 3` evaluated as `(5 + 2) - 3`

- For exponentiation `\*\*`, **associativity is right-to-left** ('RL'):

- Example: `2 \*\* 3 \*\* 2` is evaluated as `2 \*\* (3 \*\* 2)` = `2 \*\* 9` = **512**, **not** `(2 \*\* 3) \*\* 2 = 64`.

- Importance of Parentheses

- Parentheses have the highest precedence and force the expression inside them to be evaluated first.

- They allow programmers to override operator precedence and associativity to enforce a specific evaluation order.

- Using parentheses improves readability and removes ambiguity.

- Example:

- Without parentheses: `5 + 2 \* 6 - 4 / 2` evaluates multiplication and division first.

- With parentheses: `((5 + 2) \* (6 - 4)) / 2` forces addition and subtraction first before multiplication and division.

- Recommended to use fully parenthesized expressions when unsure about operators' precedence and associativity.

Section 3: Example Code and Use Cases

Example 1: Simple addition

```
x = 4 + 3 + 5  
print(x) # Output: 12
```

Explanation: Evaluates left to right, $(4+3)=7$, then $7+5=12$.

Example 2: Multiplication vs Addition precedence

```
x = 2 + 3 * 5  
print(x) # Output: 17
```

Explanation: Multiplication has higher precedence, so $3*5=15$ first, then $2 + 15 = 17$.

Example 3: Using parentheses to change precedence

```
x = (2 + 3) * 5  
print(x) # Output: 25
```

Explanation: Parentheses cause addition before multiplication: $(2+3)=5$, $5*5=25$.

Example 4: Multiple operators with same precedence and left-to-right associativity

```
x = 5 + 2 * 3 - 8 / 2  
print(x) # Output: 7.0
```

Step-by-step:

- Multiplication and division first due to higher precedence:

- $2 * 3 = 6$

- $8 / 2 = 4.0$

- Then addition and subtraction left to right:

- $5 + 6 = 11$

- $11 - 4.0 = 7.0$

Example 5: Exponentiation associativity right to left

```
x = 2 ** 3 ** 2
print(x) # Output: 512
```

Explanation: Evaluated as $2^{(3^2)} \rightarrow 2^{9} = 512$.

Example 6: Fully parenthesized expression

```
x = (((5 + 2) * (6 - 4)) / 2)
print(x) # Output: 7.0
```

Explanation: Everything is explicitly grouped:

- $(5 + 2) = 7$
- $(6 - 4) = 2$
- $7 * 2 = 14$
- $14 / 2 = 7.0$

Section 4: Key Takeaways

- Expressions combine operands and operators to compute values.
- Operator **precedence** determines which operations execute first in expressions.
- Operator **associativity** defines the order of evaluation when operators have the same precedence.
- Multiplication, division, and modulo have higher precedence than addition and subtraction.
- Exponentiation `**` has right-to-left associativity, unlike other arithmetic operators.
- Parentheses have the highest precedence, allowing explicit control over evaluation order.
- Writing **fully parenthesized expressions** is a good practice to avoid ambiguity and logic errors.
- Knowing precedence and associativity is crucial—mistakes here lead to wrong results and bugs.

- Use parentheses liberally when unsure to ensure correct expression evaluation in programs.